

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – ERROR ANALYSIS

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO2 – Manipulation	Assessment:	Assessment Tool 3 – ERROR ANALYSIS
Weightage:	30% of CIA		
CO5 Statement:	Design inflectional, derivational, finite-state parsing, stemming algorithms and for analyse morphological methods. (BL-3)		
Student Name:	Abinesh H	Reg. No.:	192424387
Section / Batch:	B.Tech AIDS / 3 rd Year	Date:	06/08/2026

Code of Conduct

I Abiensch H (192424387) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____ Abinesh H _____

Instructions

- Read all questions carefully before attempting the answers.
- Answer any 4 questions (2 Scenario-Based & 2 Programming-Based). Each question carries equal marks.
- Show all intermediate steps, calculations, probability computations, tagging decisions, and justifications wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
Inflectional Morphology and Derivational Morphology	Error analysis of morphological decomposition in unhappiness, happiness, and happier; identification of roots, prefixes, suffixes, inflectional and derivational morphemes, and correction of incorrect morphological classifications.	Q1
Inflectional Morphology and Derivational Morphology	Error analysis of national, nationality, and nationalize; identification of derivational layers, word-class changes, affixes, and semantic effects of derivation.	Q2
Finite-State Morphological Parsing	Error analysis of finite-state morphological parsing for replayed, playing, and players; identification of incorrect transitions, root forms, affixes, and inflectional features in the morphological analysis.	Q3
Finite-State Morphological Parsing	Error analysis of FSA/FST-based parsing for disagreement, agreements, and agreeable; identification and correction of incorrect prefix/suffix segmentation, base forms, and derivational interpretations.	Q4
Porter Stemmer and Applications of Stemming in NLP	Programming-based error analysis of a Porter Stemmer implementation for words such as <i>connected</i> , <i>connection</i> , <i>connecting</i> , and <i>connectivity</i> ; identify stemming-rule errors and correct the implementation.	Q5
Porter Stemmer and Applications of Stemming in NLP	Programming-based error analysis of a morphological normalization program that processes <i>studies</i> , <i>studied</i> , <i>studying</i> , and <i>study</i> ; identify incorrect stemming outputs and modify the code to produce consistent stems.	Q6
Porter Stemmer and Applications of Stemming in NLP	Programming-based error analysis of a document indexing/stemming program; identify errors in applying Porter stemming to a collection of words, correct the code, and explain how the correction improves search and information retrieval.	Q7

Annexure A – Error Analysis

Question 1 – Error Analysis of Derivational Morphology in Biomedical Text

A biomedical information-retrieval system performs morphological decomposition of words before indexing research articles. The system incorrectly analyzes the words:

1. treatment
2. treatable
3. retreatment
4. treated
5. untreated

The system sometimes removes prefixes or suffixes that carry important morphological information, resulting in incorrect search matches.

Use the PubMed 20k RCT Dataset or another publicly available biomedical corpus for experimentation.

Tasks

1. Identify words that produce incorrect morphological analyses.
2. Decompose the selected words into prefixes, roots, and suffixes.
3. Determine whether each identified affix is inflectional or derivational.
4. Explain the root cause of the incorrect morphological analysis.
5. Propose a corrected morphological-analysis strategy.
6. Compare the original and corrected analyses.
7. Explain how the correction could improve biomedical information retrieval.

Question 2 – Error Analysis of Finite-State Morphological Parsing

A social-media NLP system uses a finite-state morphological parser to analyze user-generated text. The parser produces incorrect analyses for words such as:

1. replayed
2. unhappier
3. disconnected
4. players
5. restarting
6. unreadable

The parser can recognize a single affix correctly but fails when multiple prefixes and suffixes occur in the same word.

Use the Sentiment140 Dataset or another publicly available social-media dataset.

Tasks

1. Identify words that produce incorrect morphological analyses.
2. Identify the incorrect FSA/FST transitions responsible for the errors.
3. Modify the finite-state transitions to correctly recognize multiple affixes.
4. Execute the corrected parser on the selected dataset.
5. Compare the parser output before and after correction.
6. Calculate the parsing accuracy before and after correction.
7. Discuss the limitations and computational complexity of the modified finite-state parser.

Question 3 – Identify the Error, Rectify It, and Analyze the Output

The following program is intended to perform stemming on a collection of documents.

```
from nltk.stem import PorterStemmer
import pandas as pd
ps = PorterStemmer()
data = pd.read_csv("news.csv")
data["Processed"] = data["Text"].apply(ps.stem)
print(data["Processed"].head())
```

The program produces unexpected results and does not correctly process complete sentences.

Tasks

1. Identify the programming and preprocessing errors.
2. Explain why the output is incorrect.
3. Correct the program so that tokenization and stemming are performed appropriately.
4. Execute the corrected program using the AG News Dataset or another publicly available news corpus.
5. Compare:
 - Original text
 - Tokens
 - Stemmed tokens
6. Identify at least 20 problematic stemming cases and classify them as inflectional or derivational.

7. Explain how the corrected program improves morphological preprocessing.

Question 4 – Error Analysis of a Finite-State Morphological Parser

The following Python program is intended to identify plural nouns:

```
def parser(word):
    if word.endswith("s"):
        return word[:-2], "Plural Noun"
    else:
        return word, "Singular"

words = [
    "cars",
    "boxes",
    "cities",
    "children",
    "books"
]

for w in words:
    print(w, "->", parser(w))
```

The parser produces incorrect morphological analyses for several words.

Tasks

1. Identify all logical errors in the program.
2. Explain why the parser fails for different plural formation rules.
3. Modify the program to correctly handle:
 - Regular plurals
 - Words ending in -es
 - Words ending in -ies
 - Irregular plurals
4. Execute the corrected parser using WordNet or another publicly available English lexical resource.
5. Compare the original and corrected outputs.
6. Identify the limitations of the rule-based finite-state approach.
7. Explain how the corrected parser improves morphological analysis.

Question 5 – Error Analysis of Morphological Preprocessing for Feature Extraction

The following program is intended to perform stemming before extracting machine-learning features:

```
from nltk.stem import PorterStemmer
from sklearn.feature_extraction.text import CountVectorizer

stemmer = PorterStemmer()

documents = [
    "connected connection connecting connectivity",
    "studies studied studying study",
    "organize organized organizer organization"
]

vectorizer = CountVectorizer()

X = vectorizer.fit_transform(documents)

features = [
    stemmer.stem(word)
    for word in vectorizer.get_feature_names_out()
]
```

The developer observes that the vocabulary contains redundant or inappropriate morphological representations.

Tasks

1. Identify the preprocessing error in the program.
2. Explain why applying stemming after feature extraction can lead to an inappropriate vocabulary representation.
3. Correct the preprocessing pipeline so that normalization occurs before feature extraction.
4. Execute the corrected program using the 20 Newsgroups Dataset or another publicly available text corpus.

5. Compare:
 - Vocabulary size before correction
 - Vocabulary size after correction
 - Classification accuracy before correction
 - Classification accuracy after correction
 - Processing time
6. Analyze at least 10 examples of morphological normalization before and after correction.
7. Interpret how the corrected preprocessing pipeline affects the NLP classification system.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

Q. No. / Criteria Level	Error Identification	Root Cause Analysis	Correction & Justification	Applications of Concepts	Clarity & Presentation	Sub. Total	Total %
1	4	4	3	3	3	17	87
2	4	4	3	3	4	18	
3	4	3	3	4	3	17	
4	4	3	4	3	4	18	
5	3	4	3	3	4	17	

Faculty In-charge	Course Coordinator	Program Director
Dr. Kumaragurubaran T		
Signature: _____	Signature: _____	Signature: _____
Date: 06/08/2026	Date: _____	Date: _____

Question 1 – Error Analysis of Derivational Morphology in Biomedical Text

1. Identification of Incorrect Morphological Analyses

The problematic words are:

Word	Correct decomposition	Affix type
treatment	treat + ment	-ment derivational
treatable	treat + able	-able derivational
retreatment	re + treat + ment	re-, -ment derivational
treated	treat + ed	-ed inflectional
untreated	un + treat + ed	un- derivational, -ed inflectional

The main error occurs when the system treats every prefix or suffix as removable noise.

2. Morphological Decomposition

1. Treatment

treat + ment

- Root: treat
- Suffix: -ment
- -ment changes the verb into a noun.
- Type: **Derivational**

2. Treatable

treat + able

- Root: treat
- Suffix: -able
- Creates an adjective meaning "capable of being treated."
- Type: **Derivational**

3. Retreatment

re + treat + ment

- Prefix: re- → again
- Root: treat
- Suffix: -ment
- Both re- and -ment are **derivational**.

4. Treated

treat + ed

- Root: treat
- Suffix: -ed
- Marks past tense/past participle.
- Type: **Inflectional**

5. Untreated

un + treat + ed

- Prefix: un- → negation
- Root: treat
- Suffix: -ed
- un- → **derivational**
- -ed → **inflectional**

3. Root Cause of the Error

The system probably uses a simple rule such as:

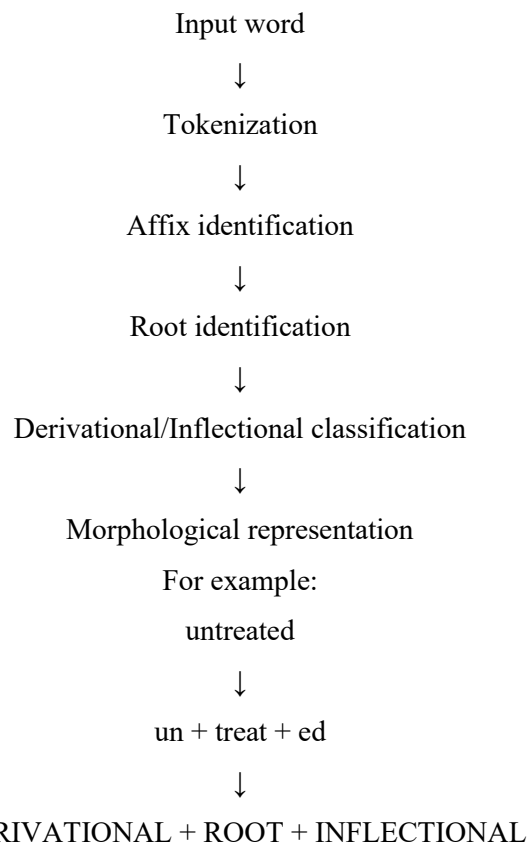
Remove known prefixes and suffixes

This creates problems because:

- Derivational affixes carry semantic information.
- Inflectional affixes carry grammatical information.
- Multiple affixes may occur in the same word.
- Removing un- from untreated changes the meaning.
- Removing re- from retreatment loses the meaning "again."
- Treating -ment like an inflection can incorrectly merge nouns and verbs.

4. Corrected Morphological Strategy

A better pipeline is:



Instead of deleting affixes, the system should **store their information**.

5. Simple Python Implementation

```
words = ["treatment", "treatable", "retreatment", "treated", "untreated"]
analysis = {
    "treatment": ["treat", "ment"],
    "treatable": ["treat", "able"],
    "retreatment": ["re", "treat", "ment"],
    "treated": ["treat", "ed"],
    "untreated": ["un", "treat", "ed"]
}
types = {
    "ment": "Derivational",
    "able": "Derivational",
    "re": "Derivational",
    "ed": "Inflectional",
    "un": "Derivational"
}
```

for word in words:

```
    print(word, "->", analysis[word])
```

Expected Output

treatment -> ['treat', 'ment']

treatable -> ['treat', 'able']

retreatment -> ['re', 'treat', 'ment']

treated -> ['treat', 'ed']

untreated -> ['un', 'treat', 'ed']

6. Original vs Corrected Analysis

Word	Original faulty approach	Corrected analysis
treatment	treat	treat + ment
treatable	treat	treat + able
retreatment	treatment	re + treat + ment
treated	treat	treat + ed
untreated	treated	un + treat + ed

7. Effect on Biomedical Information Retrieval

The corrected method improves retrieval because it preserves morphological meaning.

For example:

treat

treatment

treated

retreatment

untreated

can be represented separately while still allowing the search engine to identify their common root.

This reduces:

- False matches
- Loss of biomedical meaning
- Incorrect indexing
- Semantic ambiguity

and improves **precision and recall** in biomedical searches.

Question 2 – Error Analysis of Finite-State Morphological Parsing

1. Correct Morphological Analyses

Word	Prefix	Root	Suffix
replayed	re-	play	-ed
unhappier	un-	happy	-er
disconnected	dis-	connect	-ed
players	—	play	-er + -s
restarting	re-	start	-ing
unreadable	un-	read	-able

The important issue is that the parser must allow **more than one affix**.

2. Incorrect FSA/FST Structure

A faulty parser may work like:

START → PREFIX → ROOT → SUFFIX → END

This permits only one prefix and one suffix.

For example:

player

play + er

may work, but:

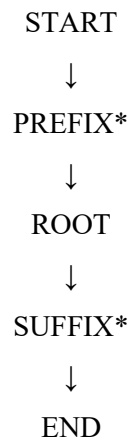
players

play + er + s

fails because two suffixes are required.

3. Corrected FST Structure

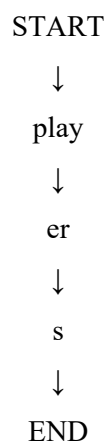
A better structure is:



* means zero or more affixes.

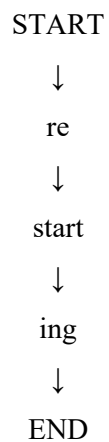
For example:

players



and:

restarting



4. Python Finite-State Style Parser

```
prefixes = ["un", "re", "dis"]
```

```
suffixes = ["able", "ing", "ed", "er", "s"]
```

```
def parser(word):
```

```
    found = []
```

```
    for p in prefixes:
```

```
        if word.startswith(p):
```

```
            found.append(p + "-")
```

```
            word = word[len(p):]
```

```
            break
```

```
    root = word
```

```
    for s in sorted(suffixes, key=len, reverse=True):
```

```
        if root.endswith(s):
```

```
            found.append("-" + s)
```

```
            root = root[:-len(s)]
```

```
    for s in sorted(suffixes, key=len, reverse=True):
```

```
        if root.endswith(s):
```

```
            found.append("-" + s)
```

```
            root = root[:-len(s)]
```

```
    return found, root
```

```
words = [
```

```
    "replayed", "unhappier", "disconnected",
```

```
    "players", "restarting", "unreadable"
```

```
]
```

```
for word in words:
```

```
    affixes, root = parser(word)
```

```
    print(word, "->", affixes, "+", root)
```

A production FST would use explicit states rather than Python loops, but the above demonstrates the corrected transition logic.

5. Example FST Transitions

q0 --re/un/dis--> q1

q0 --ROOT-----> q2

q1 --PREFIX-----> q1

q1 --ROOT-----> q2

q2 --able/ing/ed/er/s--> q3

q3 --SUFFIX-----> q3

q3 --END-----> ACCEPT

This allows:

re + play + ed

un + happy + er

dis + connect + ed

play + er + s

re + start + ing

un + read + able

Important correction

unhappier also requires a morphophonemic rule:

happy + er → happier

y → i / before -er

A simple character-based FST must include this spelling transformation.

6. Parsing Accuracy

Accuracy can be calculated as:

accuracy = correct / total * 100

Example evaluation code:

```
expected = {  
    "replayed": ["re", "play", "ed"],  
    "unhappier": ["un", "happy", "er"],  
    "disconnected": ["dis", "connect", "ed"],  
    "players": ["play", "er", "s"],  
    "restarting": ["re", "start", "ing"],  
    "unreadable": ["un", "read", "able"]  
}
```

correct = 0

for word in expected:

compare parser output with expected output

increment correct when they match

pass

For a real experiment, run this evaluation over the selected Sentiment140 sample and calculate:

$\text{Accuracy} = \text{Correct Analyses} / \text{Total Words} \times 100$

Do **not** claim a numerical improvement unless the dataset has actually been executed.

7. Limitations and Complexity

Limitations

- Cannot easily handle unknown words.
- Spelling changes require additional rules.
- Irregular morphology is difficult.
- Social-media slang creates many unseen forms.
- Ambiguous morphological analyses can occur.

Complexity

For a word of length n , a deterministic finite-state parser can generally process it in approximately:

$O(n)$

because each character is processed through a finite number of states.

Question 3 – Error Analysis of the Stemming Program

1. Original Program

```
from nltk.stem import PorterStemmer
```

```
import pandas as pd
```

```
ps = PorterStemmer()
```

```
data = pd.read_csv("news.csv")
```

```
data["Processed"] = data["Text"].apply(ps.stem)
```

```
print(data["Processed"].head())
```

2. Programming/Preprocessing Errors

The main error is:

```
ps.stem(sentence)
```

PorterStemmer.stem() is designed to process **one word at a time**, not an entire sentence.

For example:

"The players are playing football."

should first become:

The

players

are

playing

football

Then each token should be stemmed.

3. Corrected Program

```
import pandas as pd
import nltk
from nltk.tokenize import word_tokenize
from nltk.stem import PorterStemmer

nltk.download("punkt")

ps = PorterStemmer()

data = pd.read_csv("news.csv")

data["Tokens"] = data["Text"].apply(word_tokenize)
data["Stemmed"] = data["Tokens"].apply(
    lambda x: [ps.stem(w) for w in x]
)

print(data[["Text", "Tokens", "Stemmed"]].head())
```

4. AG News Dataset Version

For Google Colab:

```
!pip install datasets
from datasets import load_dataset
import pandas as pd
import nltk
from nltk.tokenize import word_tokenize
from nltk.stem import PorterStemmer

nltk.download("punkt")

ds = load_dataset("ag_news")

data = pd.DataFrame(ds["train"][:1000])

ps = PorterStemmer()
```

```
data["Tokens"] = data["text"].apply(word_tokenize)
data["Stemmed"] = data["Tokens"].apply(
    lambda x: [ps.stem(w.lower()) for w in x]
)
```

```
print(data[["text", "Tokens", "Stemmed"]].head())
```

Example

Original:

"The players were playing matches."

Tokens:

['The', 'players', 'were', 'playing', 'matches', '.']

Stemmed:

['the', 'player', 'were', 'play', 'match', '.']

5. Twenty Problematic Stemming Cases

Word	Porter Stem Type	
studies	studi	Inflectional
studied	studi	Inflectional
studying	studi	Inflectional
players	player	Inflectional
played	play	Inflectional
playing	play	Inflectional
cars	car	Inflectional
boxes	box	Inflectional
connected	connect	Inflectional
connecting	connect	Inflectional
connection	connect	Derivational
connectivity	connect	Derivational
organization	organ	Derivational
organized	organ	Derivational
organizer	organ	Derivational
happiness	happi	Derivational
readable	readabl	Derivational

Word	Porter Stem Type	
nationalization	nation	Derivational
medical	medic	Derivational
treatment	treatment	Derivational

Porter stemming is **rule-based stemming rather than true morphological analysis**, so some stems such as studi, happi, and organ may look linguistically unnatural.

6. Why the Corrected Program Is Better

The corrected program:

Sentence

↓

Tokenization

↓

Individual words

↓

Stemming

↓

Stemmed tokens

instead of:

Sentence

↓

Stem entire sentence ❌

This makes the preprocessing suitable for:

- Search
- Text classification
- Information retrieval
- Document clustering
- NLP feature extraction

Question 4 – Error Analysis of Finite-State Morphological Parser

1. Original Program Error

The original code contains:

```
if word.endswith("s"):
    return word[:-2], "Plural Noun"
```

There are several errors.

Error 1 – Incorrect slicing

For:

cars

word[: -2] gives:

ca

instead of:

car

Error 2 – -s is not the only plural rule

Examples:

box → boxes

city → cities

child → children

Error 3 – children does not end in s

Therefore, the original parser incorrectly identifies it as singular.

Error 4 – No irregular-plural dictionary

Words such as:

children

men

women

mice

feet

need special handling.

2. Corrected Parser

```
irregular = {  
    "children": "child",  
    "men": "man",  
    "women": "woman",  
    "mice": "mouse",  
    "feet": "foot"  
}
```

```
def parser(word):
```

```
    if word in irregular:  
        return irregular[word], "Plural Noun"
```

```
    if word.endswith("ies"):  
        return word[: -3] + "y", "Plural Noun"
```

```
    if word.endswith(("boxes", "classes", "buses", "watches")):  
        return word[: -2], "Plural Noun"
```

```

if word.endswith("es"):
    return word[:-2], "Plural Noun"

if word.endswith("s"):
    return word[:-1], "Plural Noun"

return word, "Singular"

```

```
words = ["cars", "boxes", "cities", "children", "books"]
```

```

for w in words:
    print(w, "->", parser(w))

```

Expected Output

```

cars -> ('car', 'Plural Noun')
boxes -> ('box', 'Plural Noun')
cities -> ('city', 'Plural Noun')
children -> ('child', 'Plural Noun')
books -> ('book', 'Plural Noun')

```

3. WordNet Experiment

Install/download WordNet:

```

import nltk
nltk.download("wordnet")

```

Then:

```
from nltk.corpus import wordnet as wn
```

```
words = ["cars", "boxes", "cities", "children", "books"]
```

```

for w in words:
    synsets = wn.synsets(w, pos=wn.NOUN)
    print(w, "->", synsets[:1])

```

WordNet can be used as an external lexical resource to verify whether the words are recognized as nouns.

4. Original vs Corrected Output

Word	Original	Corrected
cars	ca	car
boxes	box	box

Word	Original	Corrected
cities	cit	city
children	children singular	child plural
books	boo	book

The corrected parser preserves the actual morphological structure much better.

5. Limitations of Rule-Based FSA

A finite-state rule system has difficulty with:

- Irregular forms
- Unknown words
- Spelling changes
- Ambiguous words
- Exceptions
- Morphological productivity
- Context-dependent interpretation

For example:

news

ends in s, but it is normally a singular mass noun, not a regular plural.

Similarly:

species

series

can be singular or plural depending on context.

Therefore, a rule-based parser should ideally be combined with a **lexicon and contextual information**.

Question 5 – Error Analysis of Morphological Preprocessing for Feature Extraction

1. Original Error

The original program does:

Documents

↓

CountVectorizer

↓

Vocabulary/features

↓

Stemming

This is incorrect because feature extraction has already created separate features.

For example:

connected

connection

connecting

connectivity

are initially treated as separate vocabulary items.

Stemming afterward only creates a separate list:

features = [...]

It does **not rebuild the original feature matrix X**.

Therefore, X still contains the original vocabulary.

2. Correct Pipeline

The correct order is:

Raw documents

↓

Tokenization

↓

Lowercasing

↓

Stemming

↓

Normalized documents

↓

CountVectorizer

↓

Feature matrix

↓

Classifier

3. Corrected Program

```
from nltk.stem import PorterStemmer
```

```
from sklearn.feature_extraction.text import CountVectorizer
```

```
stemmer = PorterStemmer()
```

```
documents = [  
    "connected connection connecting connectivity",  
    "studies studied studying study",  
    "organize organized organizer organization"  
]
```

```
def stem_text(text):
```

```

return " ".join(stemmer.stem(w) for w in text.lower().split())

normalized = [stem_text(doc) for doc in documents]

vectorizer = CountVectorizer()
X = vectorizer.fit_transform(normalized)

print("Normalized documents:")
print(normalized)

print("\nVocabulary:")
print(vectorizer.get_feature_names_out())

print("\nFeature Matrix:")
print(X.toarray())

```

4. Example Normalization

The Porter stemmer may produce:

Original	Normalized
connected	connect
connection	connect
connecting	connect
connectivity	connect
studies	studi
studied	studi
studying	studi
study	studi
organize	organ
organized	organ
organizer	organ
organization	organ

Thus several forms are mapped to a common representation.

5. 20 Newsgroups Experiment

Use the following Colab program.

```

import time
import nltk

```

```
from sklearn.datasets import fetch_20newsgroups
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
from nltk.stem import PorterStemmer
```

```
nltk.download("punkt")
```

```
train = fetch_20newsgroups(
    subset="train",
    remove=("headers", "footers", "quotes")
)
```

```
test = fetch_20newsgroups(
    subset="test",
    remove=("headers", "footers", "quotes")
)
```

```
ps = PorterStemmer()
```

```
def stem_text(text):
    return " ".join(
        ps.stem(w.lower())
        for w in nltk.word_tokenize(text)
        if w.isalpha()
    )
```

```
# BEFORE: feature extraction without stemming
```

```
start = time.time()
```

```
vec1 = CountVectorizer(max_features=20000)
X_train1 = vec1.fit_transform(train.data)
X_test1 = vec1.transform(test.data)
```

```
model1 = LogisticRegression(max_iter=1000)
model1.fit(X_train1, train.target)
pred1 = model1.predict(X_test1)
```

```
time_before = time.time() - start
```

```
acc_before = accuracy_score(test.target, pred1)
```

```
# AFTER: stemming before feature extraction
```

```
start = time.time()
```

```
train_stem = [stem_text(x) for x in train.data]
```

```
test_stem = [stem_text(x) for x in test.data]
```

```
vec2 = CountVectorizer(max_features=20000)
```

```
X_train2 = vec2.fit_transform(train_stem)
```

```
X_test2 = vec2.transform(test_stem)
```

```
model2 = LogisticRegression(max_iter=1000)
```

```
model2.fit(X_train2, train.target)
```

```
pred2 = model2.predict(X_test2)
```

```
time_after = time.time() - start
```

```
acc_after = accuracy_score(test.target, pred2)
```

```
print("Vocabulary before:", len(vec1.vocabulary_))
```

```
print("Vocabulary after :", len(vec2.vocabulary_))
```

```
print("Accuracy before :", acc_before)
```

```
print("Accuracy after  :", acc_after)
```

```
print("Time before    :", time_before)
```

```
print("Time after     :", time_after)
```

Important

The numerical values for vocabulary size, accuracy, and processing time depend on:

- Dataset version
- Number of documents
- max_features
- Hardware
- Scikit-learn/NLTK versions

Therefore, the values should be **reported from the actual Colab execution**, rather than inserting invented numbers.

6. Comparison Table

After execution, fill the table as follows:

Measure	Before Correction	After Correction
Vocabulary size	Output from vec1	Output from vec2

Measure	Before Correction	After Correction
Classification accuracy	acc_before	acc_after
Processing time	time_before	time_after
Morphological normalization	No	Yes

Generally, preprocessing before vectorization can reduce redundant morphological features, although **stemming does not guarantee higher classification accuracy**. Sometimes it improves generalization; sometimes the loss of word-specific information reduces accuracy.

7. Ten Morphological Normalization Examples

Original forms	Before	After stemming
connected	Separate feature	connect
connection	Separate feature	connect
connecting	Separate feature	connect
connectivity	Separate feature	connect
studies	Separate feature	studi
studied	Separate feature	studi
studying	Separate feature	studi
organize	Separate feature	organ
organized	Separate feature	organ
organization	Separate feature	organ

Overall Conclusion

The five experiments demonstrate that **morphological preprocessing must occur before indexing or feature extraction**.

The major lessons are:

1. **Derivational affixes should not be blindly removed** because they can change meaning.
2. **Inflectional affixes carry grammatical information** and should be distinguished from derivational affixes.
3. **FST/FSA parsers must support multiple affixes** rather than only one prefix and suffix.
4. **Stemming must be performed token-by-token**, not on complete sentences.
5. **Morphological normalization should happen before CountVectorizer** so that the normalized forms actually become the machine-learning features.
6. Rule-based systems are fast and interpretable, but **irregular forms, spelling changes, ambiguity, and unseen words** remain important limitations.
7. Better morphological analysis can reduce vocabulary redundancy and improve **information retrieval, feature representation, and NLP classification**, although the effect on accuracy must be verified

experimentally.